Assignment - 2

Q Principles in practice : Microservices and event - driven architecture in E-commerce.

→ Scenario - Suppose we are developing an online shopping website named ShopEasy, where customers can:
- Browse products, place orders, make payments, recieve notifications, track deliveries.

1. Microservices architecture - It splits the large application into small, standalone services. Each service performs a single task and can operate independently.

In shopeasy, we develop services such as:

i) Product service - Manage product listings and inventory
ii) Order service - Manages placing and tracking orders.
iii) Payment service - manages money transactions.
iv) Shipping service - Handles delivery.
v) Notification service - sends notifications through email/SMS

Why its useful:

1) Simple to scale (expand) individual services.
2) Simpler to test, debug and deploy individual components separately.
3) Teams can work on multiple services simultaneously.

2. Event - Driven architecture - Services communicate with one another by sending and recieving messages.

How it works in ShopEasy:

Customer orders → OrderCreated event is fired.
Payment service listens and makes payment.

On success → PaymentSuccessful event is fired.
Shipping service listens and initiates delivery. Notification service sends email / SMS at every step.

3. SOLID Principles - Broken down simply

i) S - Single responsibility
Every service only performs one task (eg - order service only deals with orders).

ii) O - Open / closed
You can introduce new functionality without modifying the current payment code.

iii) L - Liskov substitution
Various kinds of users can utilize the same order system without issues.

iv) I - Interface segregation
Services only reference the parts they require.

v) D - Dependency inversion
Services rely on event types, not on how others are constructed internally.

4. DRY and KISS Principles - Shared libraries contain common code (such as logging, error handling). Events such as OrderCreated are created once and used by all services.

KISS (Keep it simple, stupid) -

1) Every service has a specific and straight forward task.

2) Don't make services too intelligent or too interdependent.

3) Make use of messages / events rather than intricate direct interdependence between services.